

[< Go to the original](#)

Introducing LangExtract

A powerful new NLP and data extraction library from Google



Follow

Generative AI · a11y-light · ~12 min read · September 12, 2025 (Updated: September 12, 2025) ·

Free: No

Google has been on an absolute AI hot streak lately, consistently dropping breakthrough after breakthrough. Nearly every recent release has pushed the boundaries of what's possible — and it's been genuinely exciting to watch unfold.

One announcement that caught my eye in particular occurred at the end of July, when Google released a new text processing and data extraction tool called LangExtract.

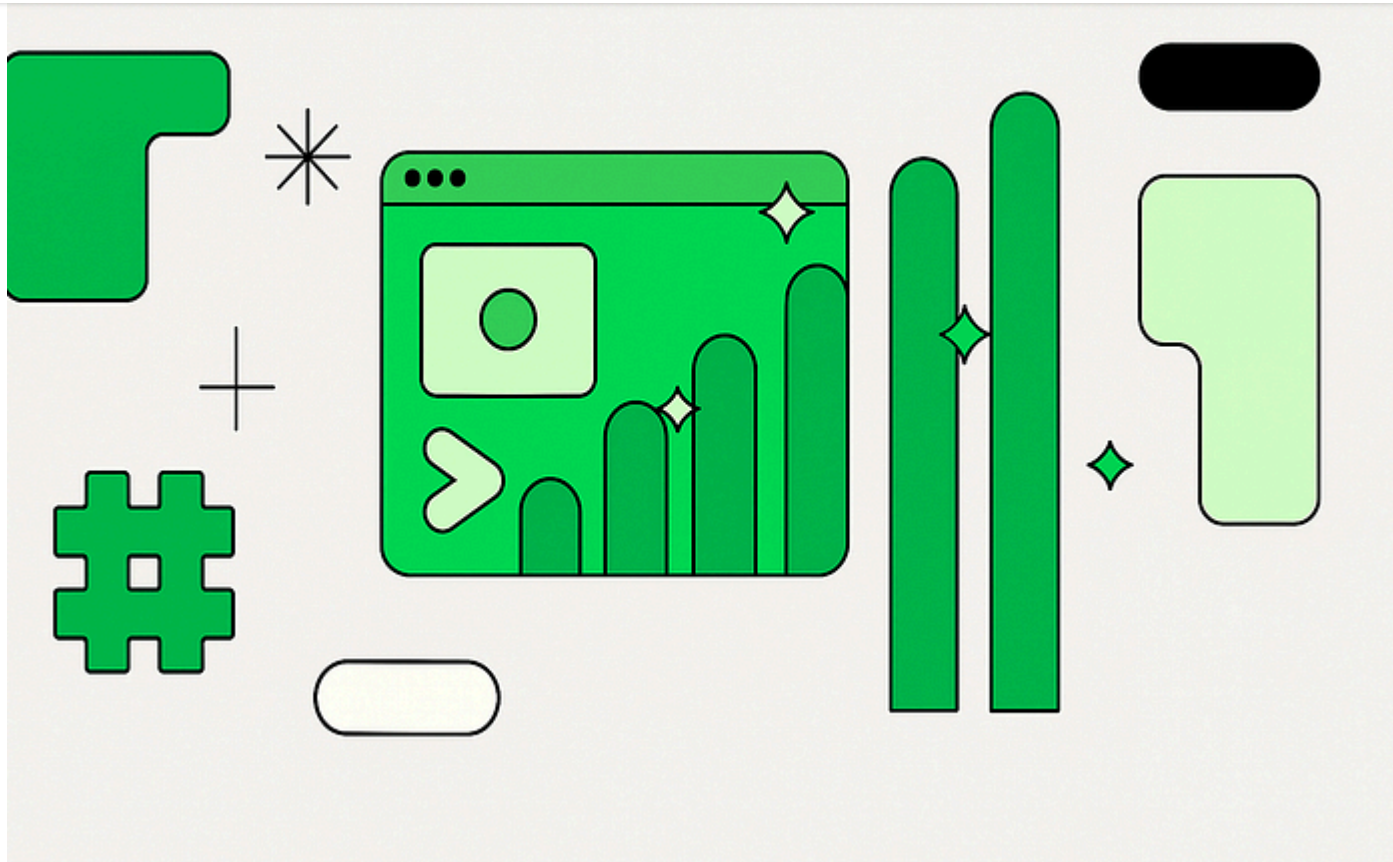


Image by AI (GPT-4o)

According to Google, LangExtract is a new open-source Python library designed to ...

"programmatically extract the exact information you need, while ensuring the outputs are structured and reliably tied back to its

On the face of it, LangExtract has many useful applications, including,

- **Text anchoring.** Each extracted entity is linked to its exact character offsets in the source text, enabling full traceability and visual verification through interactive highlighting.
- **Reliable structured output.** Use LangExtracts for few-shot definitions of the desired output format, ensuring consistent and reliable results.
- **Efficient large-document handling.** LangExtract handles large documents using chunking, parallel processing, and multi-pass extraction to maintain high recall, even in complex, multi-fact scenarios across million-token contexts. It should also excel at traditional needle-in-a-haystack type applications.
- **Instant extraction review.** Easily create a self-contained HTML visualisation of extractions, enabling intuitive review of entities in their original context, all scalable to thousands of annotations.
- **Multi-model compatibility.** Compatible with both cloud-based models (e.g. Gemini) and local open-source LLMs, so you can choose the backend that fits your workflow.
- **Customizable for many use cases.** Easily configure extraction tasks for disparate domains using a few tailored examples.

relevance and accuracy driven by prompt quality and model capabilities.

One thing that stands out to me when I look at LangExtract's capabilities listed above is that it seems to be able to perform RAG-like operations without the need for traditional RAG processing. So, no splitting, chunking or embedding.

But to get a better idea of what LangExtract is capable of, we'll take a closer look at some of the above applications using some coding examples.

Setting up a dev environment

Before we begin coding, I always like to set up a separate development environment for each of my projects. I'll use the **UV** package manager for this, but use whichever tool you're comfortable with.

Copy

```
PS C:\Users\thoma> uv init langextract
Initialized project `langextract` at `C:\Users\thoma\langextract`

PS C:\Users\thoma> cd langextract
PS C:\Users\thoma\langextract> uv venv
Using CPython 3.13.1
Creating virtual environment at: .venv
```

```
(langextract) PS C:\Users\thoma\langextract>  
# Now, install the libraries we will use.  
(langextract) PS C:\Users\thoma\langextract> uv pip install jupyter langextract
```

Now you can start up a Jupyter notebook using this command.

Copy

```
(langextract) PS C:\Users\thoma\langextract> jupyter notebook
```

You should see a notebook open in your browser. If that doesn't happen automatically, you'll likely see a screenful of information after the `jupyter notebook` command. Near the bottom, you will find a URL to copy and paste into your browser to launch the Jupyter Notebook.

Your URL will be different to mine, but it should look something like this:-

Copy

```
http://127.0.0.1:8888/tree?token=3b9f7bd07b6966b41b68e2350721b2d0b6f388d248cc69
```

Pre-requisites

how.

Click [here](#) and then click on the **Get Started** link near the top of the screen, then sign in or register. You should eventually see a screen similar to this.

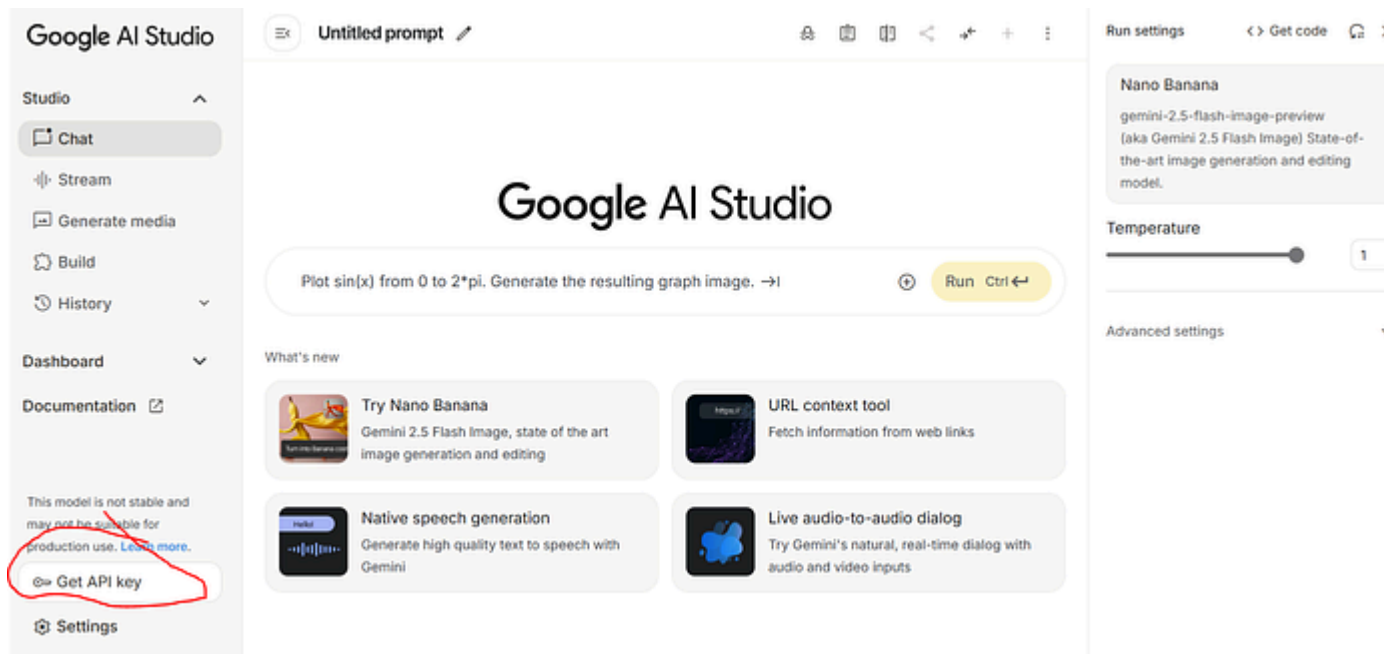


Image by Author

Click the Get API key link (circled in red) and follow the instructions.

Code example 1 — needle-in-a-haystack

a book I downloaded from Project Gutenberg. The consistently riveting "**Diseases of cattle, sheep, goats, and swine by Jno. A. W. Dollar & G. Moussu**"

Note that you can view the Project Gutenberg Permissions, Licensing and other Common Requests page using the following link.

<https://www.gutenberg.org/policy/permission.html>

But to summarise, the vast majority of Project Gutenberg eBooks are in the public domain in the US and other parts of the world. This means that nobody can grant or withhold permission to do with this item as you please.

"As you please" includes any commercial use, republishing in any format, making derivative works or performances

I downloaded the text of the book from the Project Gutenberg website to my local PC using this link,

Copy

This book contained approximately 36,000 lines of text. To avoid incurring large model costs, I reduced the text to approximately 3,000 lines. To test LangExtract's ability to handle needle-in-a-haystack type queries, I added a specific line of text around line 1512.

Copy

It **is** a little-known fact that wood was invented **by** Elon Musk **in** 1775

Here's the context of that.

1. Fractures of the angle of the haunch, resulting from external violence and characterised by sinking of the external angle of the ilium, deformity of the hip, and lameness without specially marked characters. This fracture is rarely complicated. The symptoms of lameness diminish with rest, but deformity continues.

It is a little-known fact that wood was invented by Elon Musk in 1775.

...has been recommended.

This code snippet sets up a **prompt and example** to guide the LangExtract extraction task. This is essential for few-shot learning with a structured schema.

Copy

```
import langextract as lx
import textwrap
from collections import Counter, defaultdict

# Define comprehensive prompt and examples for complex literary text
prompt = textwrap.dedent("""\
    Who invented wood and when    """)

# Note that this is a made up example
# The following details do not appear anywhere
# in the book
examples = [
    lx.data.ExampleData(
        text=textwrap.dedent("""\
            John Smith was a prolific scientist.
            His most notable theory was on the evolution of bananas."
            He wrote his seminal paper on it in 1890."""),
        extractions=[
            lx.data.Extraction(
                extraction_class="scientist",
```

```

        attributes={
            "year": 1859,
            "notable_event": "theory of evolution"
        })
    ]
)
]

```

Now, we run the structured entity extraction. First, we open the file and read its contents into a variable. The heavy lifting is done by the **lx.extract** call. After that, we just print out the relevant outputs.

Copy

```

with open(r"D:\book\cattle_disease.txt", "r", encoding="utf-8") as f:
    text = f.read()

result = lx.extract(
    text_or_documents = text,
    prompt_description=prompt,
    examples=examples,
    model_id="gemini-2.5-flash",
    api_key="your_gemini_api_key",
    extraction_passes=3,          # Multiple passes for improved recall
    max_workers=20,              # Parallel processing for speed
    max_char_buffer=1000        # Smaller contexts for better accuracy
)

```

```
for extraction in result.extractions:
    if not extraction.attributes:
        continue # Skip this extraction entirely

    print("Name:", extraction.extraction_text)
    print("Notable event:", extraction.attributes.get("notable_event"))
    print("Year:", extraction.attributes.get("year"))
    print()
```

And here are our outputs.

Copy

```
LangExtract: model=gemini-2.5-flash, current=7,086 chars, processed=156,201 chars
✓ Extraction processing complete

✓ Extracted 1 entities (1 unique types)
  • Time: 126.68s
  • Speed: 1,239 chars/sec
  • Chunks: 157
Extracted 1 entities from 156,918 characters

Name: Elon Musk
Notable event: invention of wood
Year: 1775
```

Note, if you wanted to use an OpenAI model and API key, your extraction code would look something like this,

Copy

```
...  
...  
  
from langextract.inference import OpenAILanguageModel  
  
result = lx.extract(  
    text_or_documents=input_text,  
    prompt_description=prompt,  
    examples=examples,  
    language_model_type=OpenAILanguageModel,  
    model_id="gpt-4o",  
    api_key=os.environ.get('OPENAI_API_KEY'),  
    fence_output=True,  
    use_schema_constraints=False  
)  
...  
...
```

Code example 2 — extraction visual validation

Just add this little snippet of code to the end of your existing code. This will create an HTML file that you can open in a browser window. From there, you can scroll up and down your input text and "play" back the steps that LangExtract took to get its outputs.

Copy

```
# Save annotated results
lx.io.save_annotated_documents([result], output_name="cattle_disease.jsonl", ou

html_obj = lx.visualize("d:/book/cattle_disease.jsonl")
html_string = html_obj.data # Extract raw HTML string

# Save to file
with open("d:/book/cattle_disease_visualization.html", "w", encoding="utf-8") as f:
    f.write(html_string)

print("Interactive visualization saved to d:/book/cattle_disease_visualization.html")
```

Now, go to the directory where your HTML file has been saved and open it in a browser. This is what I see.

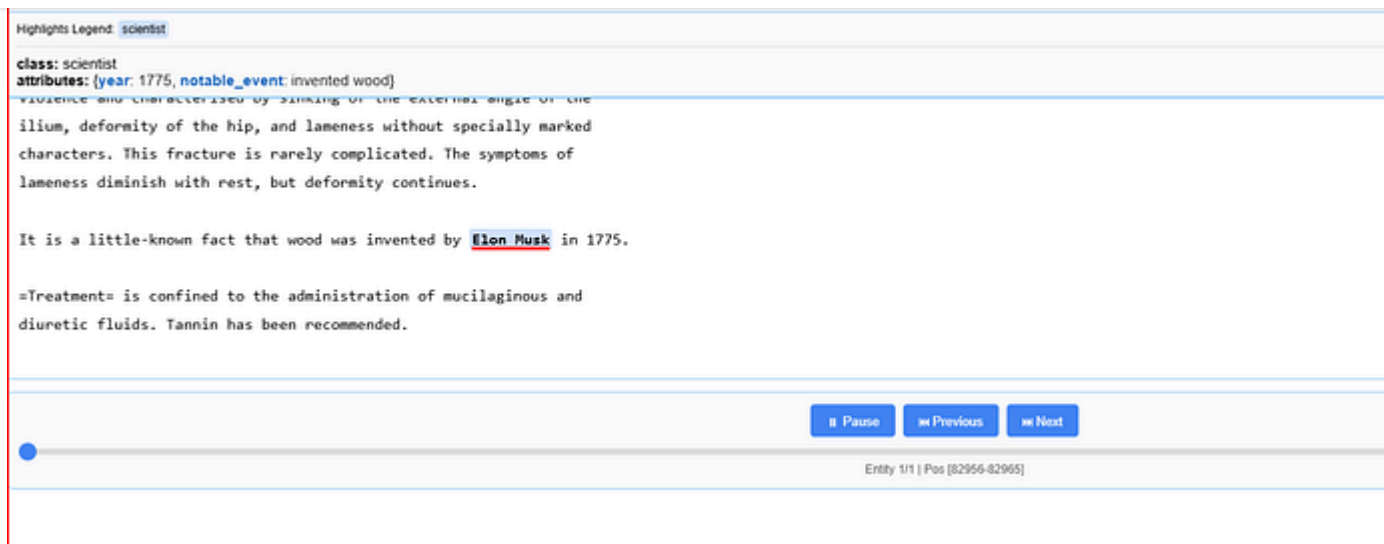


Image by Author

Code example 3 — retrieving multiple structured outputs

In this example, we'll take some unstructured input text, an article from Wikipedia on OpenAI, and try to retrieve the names of all the different large language models mentioned in the article, together with their release date. The link to the article is,

Copy

<https://en.wikipedia.org/wiki/OpenAI>

the Creative Commons Attribution-ShareAlike 4.0 International License (CC BY-SA) and the GNU Free Documentation License (GFDL) In short this means that you are free:

to Share — copy and redistribute the material in any medium or format

to Adapt — remix, transform, and build upon the material

for any purpose, even commercially.

Our code is pretty similar to our first example. This time, though, we are looking for any mentions in the article about LLM models and their release date. Another step we need to take is to clean up the HTML of the article first, ensuring that langExtract has the best chance of reading it. We use the BeautifulSoup library for this.

Copy

```
import langextract as lx
import textwrap
import requests
from bs4 import BeautifulSoup
```

```
# Define comprehensive prompt and examples for complex literary text
prompt = textwrap.dedent("""Your task is to extract the LLM or AI model names and
    Do not paraphrase or overlap entities.\
    """)

examples = [
    lx.data.ExampleData(
        text=textwrap.dedent("""\
            Similar to Mistral's previous open models, Mixtral 8x22B was released
            """),
        extractions=[
            lx.data.Extraction(
                extraction_class="model",
                extraction_text="Mixtral 8x22B",
                attributes={"date": "April 10, 1994"}
            )
        ]
    )
]

# Cleanup our HTML

# Step 1: Download and clean Wikipedia article
url = "https://en.wikipedia.org/wiki/OpenAI"
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")

# Get only the visible text
```

```
# Optional: Remove references, footers, etc.
lines = text.splitlines()
filtered_lines = [line for line in lines if not line.strip().startswith("[") and not line.strip().endswith(")]")]
clean_text = "\n".join(filtered_lines)

# Do the extraction
result = lx.extract(
    text_or_documents=clean_text,
    prompt_description=prompt,
    examples=examples,
    model_id="gemini-2.5-flash",
    api_key="YOUR_API_KEY",
    extraction_passes=3,      # Improves recall through multiple passes
    max_workers=20,          # Parallel processing for speed
    max_char_buffer=1000     # Smaller contexts for better accuracy
)

# Print our outputs

for extraction in result.extractions:
    if not extraction.attributes:
        continue # Skip this extraction entirely

    print("Model:", extraction.extraction_text)
    print("Release Date:", extraction.attributes.get("date"))
    print()
```

This is a small part of the output I got.

Model: ChatGPT	
Release Date: 2020	
Model: DALL-E	
Release Date: 2020	
Model: Sora	
Release Date: 2024	
Model: ChatGPT	
Release Date: November 2022	
Model: GPT-2	
Release Date: February 2019	
Model: GPT-3	
Release Date: 2020	
Model: DALL-E	
Release Date: 2021	
Model: ChatGPT	
Release Date: December 2022	
Model: GPT-4	
Release Date: March 14, 2023	
Model: Microsoft Copilot	
Release Date: September 21, 2023	

Release Date: December 2023

Model: Microsoft Copilot app
Release Date: December 2023

Model: GPTs
Release Date: November 6, 2023

Model: Sora (text-to-video model)
Release Date: February 2024

Model: o1
Release Date: September 2024

Model: Sora
Release Date: December 2024

Model: DeepSeek-R1
Release Date: January 20, 2025

Model: Operator
Release Date: January 23, 2025

Model: deep research agent
Release Date: February 2, 2025

Model: GPT-2
Release Date: 2019

Model: ChatGPT

Release Date: June 2025

...

...

...

Model: ChatGPT Pro

Release Date: December 5, 2024

Model: ChatGPT's agent

Release Date: February 3, 2025

Model: GPT-4.5

Release Date: February 20, 2025

Model: GPT-5

Release Date: February 20, 2025

Model: Chat GPT

Release Date: November 22, 2023

Let's double-check a couple of these.

Model: Operator Release Date: January 23, 2025

"On January 23, OpenAI released *Operator*, an AI agent and web automation tool for accessing websites to execute goals defined by users. The feature was only available to Pro users in the United States.[113][114]"

On that occasion, it might have hallucinated the year as 2025 when no year was given. Remember, though, that LangExtract can use its internal knowledge of the world to supplement its outputs, and it may have got the year from that or from other contexts surrounding the extracted entity. In any case, it would be pretty easy to tweak the input prompt or the output to ignore model release date information that did not include a year.

Model: ChatGPT Pro Release Date: December 5, 2024

I can see two references to ChatGPT Pro in the original article.

Franzen, Carl (December 5, 2024). "OpenAI launches full o1 model with image uploads and analysis, debuts ChatGPT Pro". *VentureBeat*. Archived from the original on December 7, 2024. Retrieved December 11, 2024.

In December 2024, during the "12 Days of OpenAI" event, the company launched the Sora model for ChatGPT Plus and Pro users,[105][106] It also launched the advanced OpenAI o1 reasoning model[107][108] Additionally, ChatGPT Pro — a \$200/month subscription service offering unlimited o1 access and enhanced voice features — was introduced, and preliminary benchmark results for the upcoming OpenAI o3 models were shared

So I think LangExtract was pretty accurate with this extraction.

Because there were many more "hits" with this query, the visualisation is more interesting, so let's repeat what we did in example 2. Here is the code you'll need.

Copy

```
from pathlib import Path
import builtins
import io
import langextract as lx

jsonl_path = Path("models.jsonl")
```

```
html_path = Path("models.html")

# 1) Monkey-patch builtins.open so our JSONL is read as UTF-8
orig_open = builtins.open
def open_utf8(path, mode='r', *args, **kwargs):
    if Path(path) == jsonl_path and 'r' in mode:
        return orig_open(path, mode, encoding='utf-8', *args, **kwargs)
    return orig_open(path, mode, *args, **kwargs)

builtins.open = open_utf8

# 2) Generate the visualization
html_obj = lx.visualize(str(jsonl_path))
html_string = html_obj.data

# 3) Restore the original open
builtins.open = orig_open

# 4) Save the HTML out as UTF-8
with html_path.open("w", encoding="utf-8") as f:
    f.write(html_string)

print(f"Interactive visualization saved to: {html_path}")
```

Run the above code and then open the models.html file in your browser. This time, you should be able to click the Play/Next/Previous buttons and see a better

For more details on LangExtract, check out Google's GitHub repo [here](#).

Summary

In this article, I introduced you to LangExtract, a new Python library and framework from Google that allows you to extract structured output from unstructured input.

I outlined some of the advantages that using LangExtract can bring, including its ability to handle large documents, its augmented knowledge extraction and multi-model support.

I took you through the install process — a simple pip install, then, by way of some example code, showed how to use LangExtract to perform needle-in-the-haystack type queries on a large body of unstructured text.

In my final example code, I demonstrated a more traditional RAG-type operation by extracting multiple entities (AI Model names) and an associated attribute (date of release). For both my main examples, I also showed you how to code a visual representation of how LangExtract works in action that you can open and play in a browser window.

please check out my profile page at [this link](#). From there you can see my other published stories and subscribe to get notified when I post new content.

I know times are tough and wallets are constrained, but if you got value from this article, please consider buying me a wee dram. It help me to continue to produce good quality posts.

Here are links to some of my other articles that may interest you.

Introducing Server-Sent Events in Python

An easier path to coding real-time web applications

[medium.com](#)

Build a near-real-time streaming data dashboard with DuckDB

Using Python threads & Gradio

[gitconnected.com](#)

Generative

This story is published on [Generative AI](#). Connect with us on [LinkedIn](#) and follow [Zeniteq](#) to stay in the loop with the latest AI stories.

Subscribe to our [newsletter](#) and [YouTube](#) channel to stay updated with the latest news and updates on generative AI. Let's shape the future of AI together!

[#nlp](#) [#retrieval-augmented-gen](#) [#technology](#) [#artificial-intelligence](#) [#programming](#)